



CS1004 – Object Oriented Programming

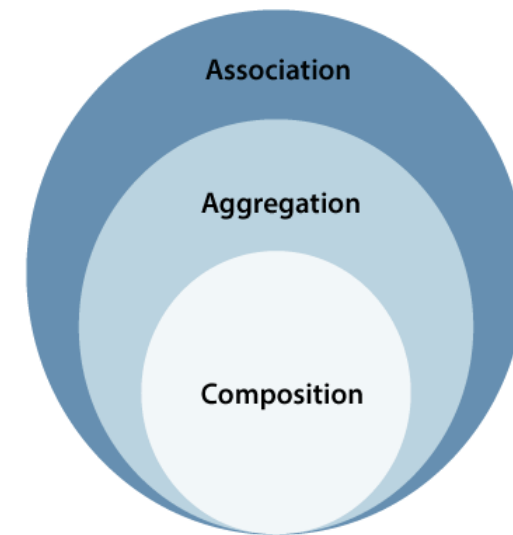
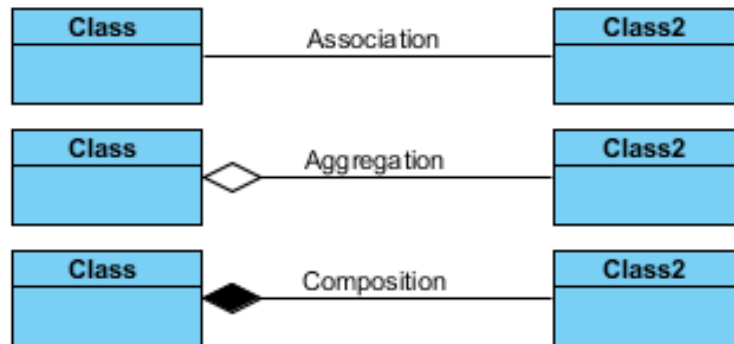
Lecture # 15
Wednesday, April 14, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

Outline

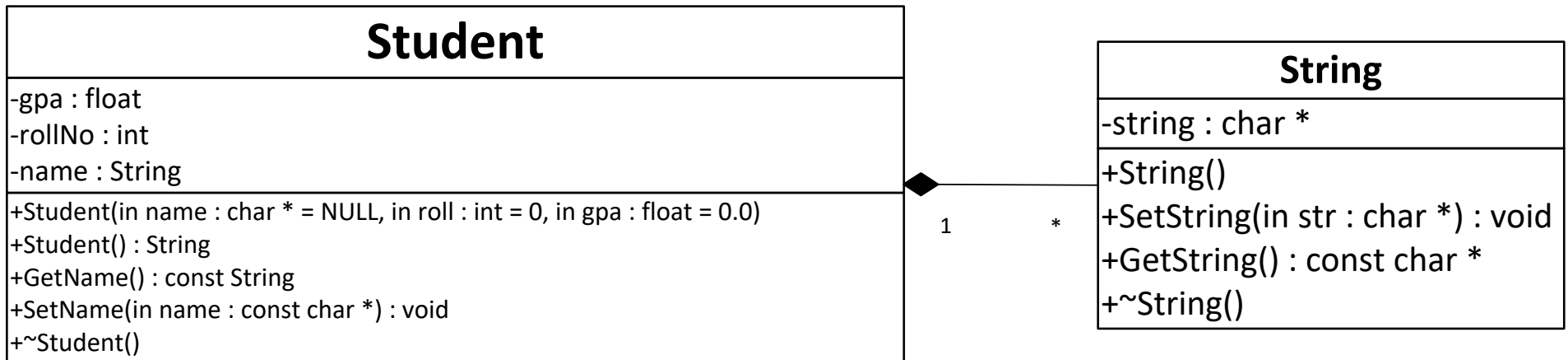
- Association
- Composition
- Aggregation
- Friend Functions
- Friend Classes

Association

- Association is general term which represents the relationship between two or more classes.
- In specific, this relationship can be either **composition** or **aggregation**.



Composition: Conceptually



Composition

- In real life objects are composed of multiple objects
- Example
 - Person: Legs, Head, Arms, etc.
 - Car: tires, steering wheel, Wind Screen etc.
- This phenomenon of Complex objects being composed of simpler one is known as **composition**
- It is also known as **HAS-A** Relationship
- Composed object cannot live without the composing object

Composition

- Car: Specific Engine
- Company: Bank Accounts
- This phenomenon of Complex objects being composed of simpler one is known as **composition**
- It is also known as **HAS-A** Relationship
- Composed object cannot live without the composing object. Engine & Bank Accounts classes are dependent on Car and Company classes.

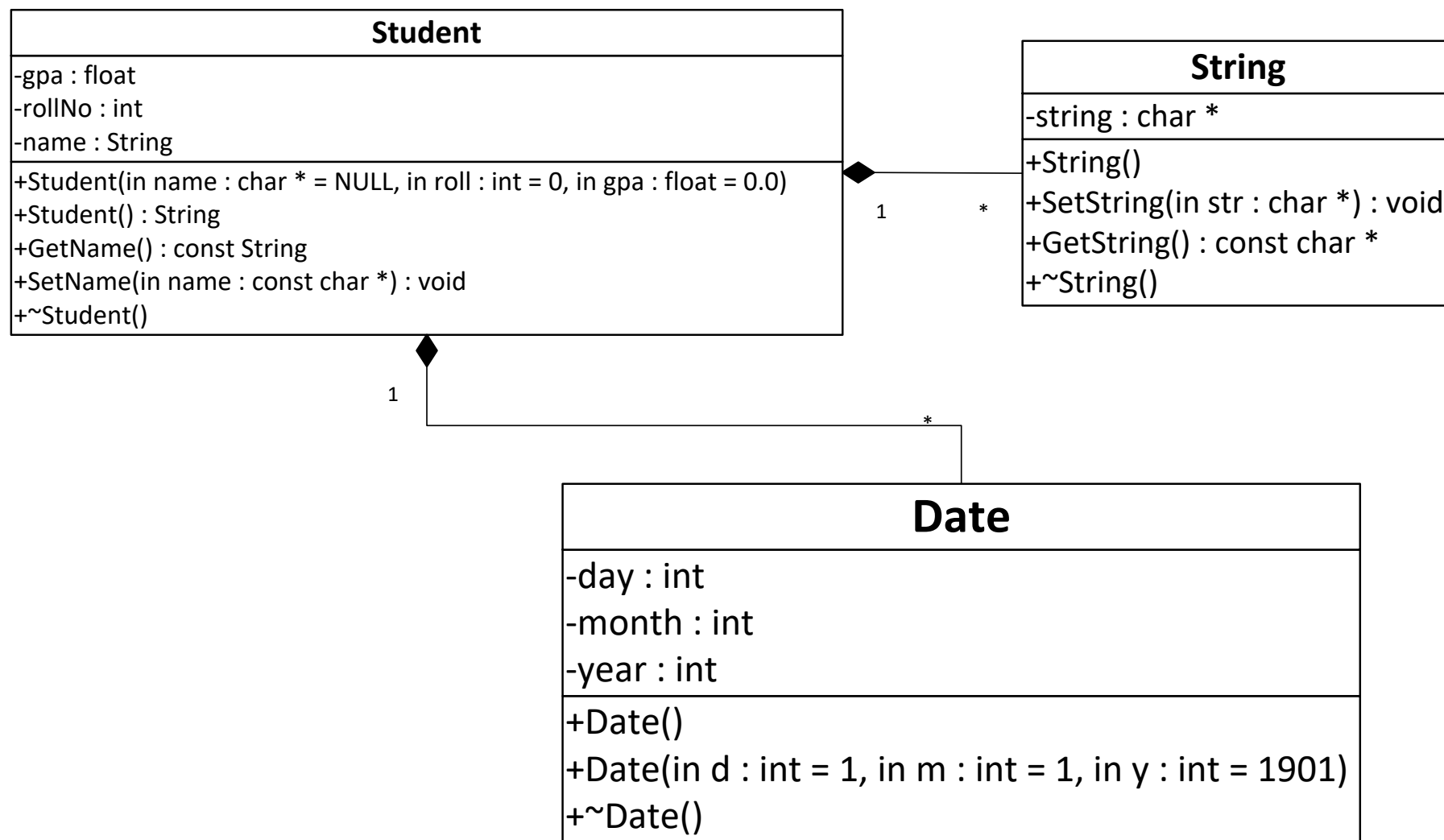
Composition

```
Student::Student(char * n, int roll, float g) {  
    cout << "Constructor:: Student..\n";  
    name.SetString(n);  
    rollNumber = roll;  
    gpa = g;  
}  
  
int main() {  
    Student aStudent("Test Student", 1024, 3.5);  
    cout << endl;  
    cout << "Name: " << aStudent.GetName() << endl;  
    return 0;  
}
```

Composition

- To assign meaningful values to the object, the function `SetString` is called explicitly in the constructor
- This is an overhead
- Sub-object name in the student class can be initialized using the constructor
- “Member initialization list” syntax is used

Composition



Composition

```
class Date {  
    int day, month, year;  
    public:  
    Date() {}  
    Date(int d = 1, int m = 1,  
        int y = 1901):day(d), month(m),  
        year(y) {}  
    ~Date() {}  
};
```

```
class Student {  
    protected:  
        int age;  
        Date birthDate;  
    public:  
        Student() {  
            cout << "\nDefault Student"; }  
        int getAge()  
        {  
            return age;  
        }  
        void setAge(int anAge) {  
            age = anAge; }  
};
```

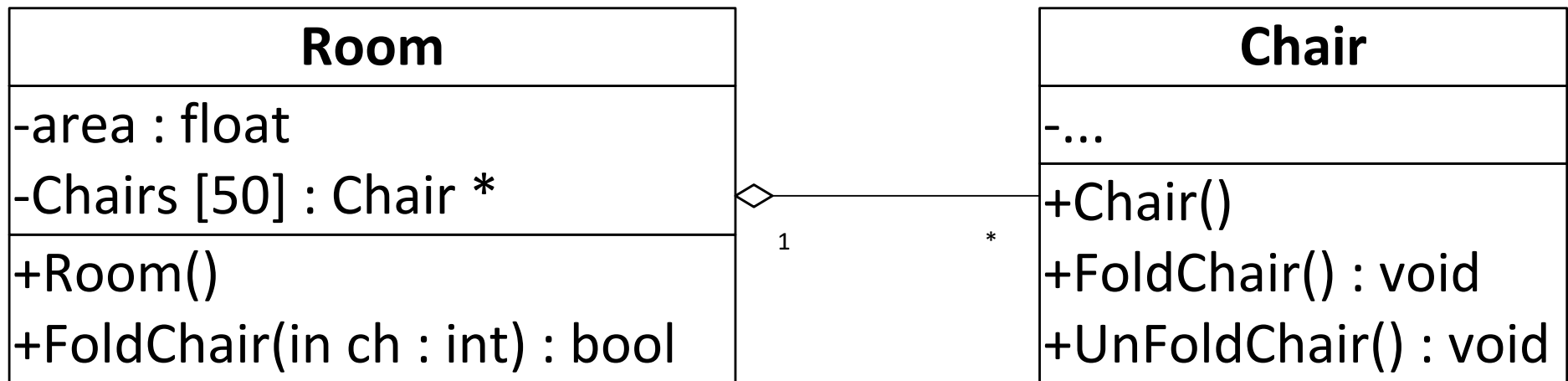
Update Student Class

```
Student(int anAge, Date bD) :age(anAge), birthDate(bD) {  
    cout << "Overloaded Constructor";  
}  
  
int main()  
{  
    Date dt(12, 12, 1998);  
    Student obj(28,dt);  
}
```

Aggregation

- It is a weak HAS_A relationship
- The composed object can live without the composing object
- Attained by creating reference or pointer of the aggregated class in main class
- Examples; **Car --- Wheels, Company --- Employees.** Here Wheel and Employees classes are **independent** of Car and Company classes.

Aggregation



Aggregation

- In aggregation, a pointer or reference to an object is created inside a class
 - The sub-object has a life that is NOT dependent on the life of its master class
- e.g:
 - Chairs can be moved inside or outside at anytime
 - When Room is destroyed, the chairs may or may not be destroyed

Aggregation

```
class Chair {  
    int numLegs;  
public:  
    Chair() {}  
    bool foldChair() { return true; }  
    bool unFoldChair() { return true; }  
};
```

```
class Room {  
private:  
    float area;  
    Chair * chairs[50];  
public:  
    Room();  
    void AddChair(Chair *, int);  
    Chair * GetChair(int);  
    bool FoldChair(int);  
};
```

```
Room::Room(){  
    for (int i = 0; i < 50; ++i)  
        chairs[i] = nullptr;  
}  
void Room::AddChair(Chair* ch, int chNo){  
    if (chNo > 0 && chNo < 50)  
        chairs[chNo] = ch;  
}  
Chair* Room::GetChair(int chNo){  
    if (chNo > 0 && chNo < 50)  
        return chairs[chNo];  
    return nullptr;  
}  
bool Room::FoldChair(int chNo){  
    if (chNo > 0 && chNo < 50)  
        return chairs[chNo]->foldChair();  
    return false;  
}  
bool Room::FoldChair(int chNo){  
    if (chNo > 0 && chNo < 50)  
        return chairs[chNo]->foldChair();  
    return false;  
}
```

Aggregation

```
int main() {  
    Chair ch1;  
    {  
        Room r1;  
        r1.AddChair(&ch1, 1);  
        r1.FoldChair(1);  
    }  
    ch1.unFoldChair();  
    return 0;  
}
```


Friend Functions and Friend Classes

Friend Functions

➡ Let us consider a class

```
class Test {  
    int x, y;  
public:  
    void func();  
};  
  
void Test::func()  
{  
    cout << x << endl  
        << y << endl;  
}
```

Friend Function

➡ Let's consider a global function now

```
void testFunc(Test t)
{
    t.x = 15; //Error
    t.y = 25; //Error
}
```

Friend Function

- To access private data members of the `class Test` we need to make the function `testFunc()` a `friend` of the `class Test`

```
class Test {  
    int x, y;  
public:  
    void func();  
    friend void testFunc(Test&);  
};  
void Test::func()  
{  
    cout << x << endl  
    << y << endl;  
}
```

- Now `testFunc` can access private data members of `class Test`

Friend Function

- ➡ Prototypes of friend functions appear in the class definition
- ➡ But friend functions are NOT member functions

Friend Functions

- Friend functions can be placed anywhere in the class without any effect
- Access specifiers don't affect friend functions or classes

```
class Test {  
    int x, y;  
    friend void testFunc1(Test&);  
public:  
    void func();  
    friend void testFunc(Test&);  
};
```

Friend Function

- ➡ The definition of testFunc

```
void testFunc(Test &t)
{
    t.x = 15;    //No Error
    t.y = 25;    //No Error
}
```

- ➡ Friend is not given in the definition of the function

Friend Functions

- ➡ If keyword friend is used in the function definition, it's a syntax error

```
//It is an error
friend void testFunc(Test &t)
{
    t.x = 15;
    t.y = 25;
}
```


Friend Classes

- ➡ Similarly, one class can also be made friend of another class:

```
class Test {  
    ...  
    friend class Test1;  
    ...  
};
```

- ➡ Member functions of class Y can access private data members of class X

Friend Classes

- ➡ **Test1** is made a friend of **Test** class. Now it can access private members of **Test** directly

```
class Test1 {  
    int x_data;  
    Test obj;  
public:  
    void setTest()  
    {  
        obj.x = 105;  
        obj.y = 205;  
    }  
};
```

Friend Class

```
int main()
{
    Test1 t;
    t.setTest();
    return 0;
}
```

Questions

